

東京 2025 LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

Adaptive Futexes

Steven Rostedt <rostedt@goodmis.org>



東京 2025
LINUX
PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Problem Statement

Locking contention can be one of the biggest factors causing performance degradation.

User space spinlocks can cause issues if the owner is sleeping (worse if the owner is preempted by the spinner).

There's no reason to spin if the owner is sleeping.

Time Slice Extension

Thomas Gleixner is working on a Time Slice extension

- Allows user space to tell the kernel it is in a critical section
- Uses rseq
- The kernel tells user space if it was given an extension
- When leaving critical section, it time was given, user space should call `rseq_slice_yield()`
- Perfect for short user space spin locks
- Does not address contention

FUTEX_SPIN

Last year André Almeida posted a patch set to implement FUTEX_SPIN

https://lore.kernel.org/lkml/20240523200704.281514-1-andrealmeid@igalia.com/

Feedback was light and André was distracted by other work and progress stopped on this patch

Using FUTEX_SPIN and Time Slice Extension

- Have FUTEX info include the process ID (or pidfd) of the owner
- The kernel can tell if the owner is still running or not
  - Only spin if the owner is still running
  - Sleep if the owner is sleeping

Considerations

- Cost of entering the kernel
  - For very short lived critical sections, time slice extension may be good enough
- For any long critical sections, spinning if owner is running may be better
- Both help lower contention